

Input:

- V . Set of nodes
- E_s . Set of scheduled edges $(v, u, \text{dep}, \text{arr})$
- E_t . Set of transfer edges (v, w, Δ)
- $\text{country}(v)$. Country associated with node v
- $R(v, b)$. Precomputed reachable country sets
- $MTT(c)$. Precomputed minimum transit times for each country
- $\text{bucket}(t)$. Time bucket mapping function
- record . Current record to beat (e.g., 13)
- rollout_freq . Number of iterations between rollouts (e.g., 1000)

Output:

- P . Sequence of labels representing a route within 24 hours

```

1: Initialize  $\text{Labels}(v) \leftarrow \emptyset$  for all  $v \in V$ 
2: Initialize priority queue  $Q \leftarrow \emptyset$  // ordered by decreasing score
3: Initialize  $\text{best\_label} \leftarrow \text{null}$ 
4:  $\text{iters} \leftarrow 0$  // Iteration counter for rollouts
5: // Initialize labels at departures
6: for all  $v \in V$  do
7:   for all scheduled edges  $(v, u, \text{dep}, \text{arr}) \in E_s$  do
8:      $L \leftarrow (u, \text{arr}, \{\text{country}(u)\}, \text{pred} = \text{null}, \text{start\_time} = \text{dep})$ 
9:      $\text{Labels}(u) \leftarrow \text{Labels}(u) \cup \{L\}$ 
10:    compute score( $L$ )
11:     $Q \leftarrow Q \cup \{L\}$ 
12: // Main loop
13: while  $Q \neq \emptyset$  do
14:   extract  $L = (v, t, C, \text{pred}, \text{start\_time})$  with maximum score from  $Q$ 
15:   // — NEW: MONTE CARLO ROLLOUT TRIGGER —
16:    $\text{iters} \leftarrow \text{iters} + 1$ 
17:   if  $\text{iters} \bmod \text{rollout\_freq} = 0$  then
18:      $L_{\text{rollout}} \leftarrow \text{Algorithm 7}(L, E_s, E_t, \text{country})$ 
19:     if  $\text{best\_label} = \text{null} \vee |L_{\text{rollout}}.C| > |\text{best\_label}.C|$  then
20:        $\text{best\_label} \leftarrow L_{\text{rollout}}$ 
21:       if  $|\text{best\_label}.C| > \text{record}$  then
22:         break // early stopping: rollout found a record-breaking path
23:       // -----
24:   // Enforce 24-hour journey limit (1440 absolute minutes)
25:   if  $t - \text{start\_time} > 1440$  then
26:     continue
27:   // Upper bound computation (Time-Budgeted)
28:    $b \leftarrow \text{bucket}(t)$ 
29:    $U \leftarrow R(v, b) \setminus C$ 
30:   Sort  $U$  as a sequence  $(c_1, c_2, \dots, c_k)$  such that  $MTT(c_1) \leq MTT(c_2) \leq \dots \leq MTT(c_k)$ 
31:    $\text{time\_spent} \leftarrow 0$ 
32:    $\text{UB} \leftarrow 0$ 

```

```

33: remaining_time  $\leftarrow$  1440 - (t - start_time)
34: for i = 1 to |U| do
35:   time_spent  $\leftarrow$  time_spent + MTT( $c_i$ )
36:   if time_spent  $\leq$  remaining_time then
37:     UB  $\leftarrow$  UB + 1
38:   else
39:     break
40: // Record-based pruning
41: if |C| + UB  $\leq$  record then
42:   continue
43: // Update best label from normal queue expansion
44: if best_label = null  $\vee$  |C| > |best_label.C| then
45:   best_label  $\leftarrow$  L
46:   if |C| > record then
47:     break // early stopping: record beaten
48: // Extend via scheduled edges
49: for all (v, u, dep, arr)  $\in E_s$  do
50:   if dep  $\geq$  t & dep - start_time  $\leq$  1440 then
51:      $t_{\text{new}} \leftarrow$  arr
52:     if  $t_{\text{new}} - \text{start\_time} > 1440$  then
53:       continue
54:      $C' \leftarrow C \cup \{\text{country}(u)\}$ 
55:      $L' \leftarrow (u, t_{\text{new}}, C', \text{pred} = L, \text{start\_time} = \text{start\_time})$ 
56:     if  $\nexists L'' \in \text{Labels}(u)$  such that
57:       ( $L''.t \leq t_{\text{new}} \wedge L''.C \supseteq C' \wedge (L''.t < t_{\text{new}} \vee L''.C \neq C')$ ) then
58:         remove all  $L'' \in \text{Labels}(u)$  such that
59:           ( $t_{\text{new}} \leq L''.t \wedge C' \supseteq L''.C \wedge (t_{\text{new}} < L''.t \vee C' \neq L''.C)$ )
60:          $\text{Labels}(u) \leftarrow \text{Labels}(u) \cup \{L'\}$ 
61:         compute score( $L'$ )
62:          $Q \leftarrow Q \cup \{L'\}$ 
63: // Extend via transfers
64: for all (v, w,  $\Delta$ )  $\in E_t$  do
65:    $t_{\text{new}} \leftarrow t + \Delta$ 
66:   if  $t_{\text{new}} - \text{start\_time} > 1440$  then
67:     continue
68:    $L' \leftarrow (w, t_{\text{new}}, C, \text{pred} = L, \text{start\_time} = \text{start\_time})$ 
69:   if  $\nexists L'' \in \text{Labels}(w)$  such that
70:     ( $L''.t \leq t_{\text{new}} \wedge L''.C \supseteq C \wedge (L''.t < t_{\text{new}} \vee L''.C \neq C)$ ) then
71:       remove all  $L'' \in \text{Labels}(w)$  such that
72:         ( $t_{\text{new}} \leq L''.t \wedge C \supseteq L''.C \wedge (t_{\text{new}} < L''.t \vee C \neq L''.C)$ )
73:        $\text{Labels}(w) \leftarrow \text{Labels}(w) \cup \{L'\}$ 
74:       compute score( $L'$ )
75:        $Q \leftarrow Q \cup \{L'\}$ 
76: // Reconstruct path
77:  $P \leftarrow$  empty sequence
78:  $L \leftarrow \text{best\_label}$ 
79: while  $L \neq \text{null}$  do
80:   prepend L to P
81:    $L \leftarrow L.\text{pred}$ 
82: return P

```
